

AI ASSISTED CODING

ASSIGNMENT-5.3

2303A51653

BATCH-28

Task 1: Privacy and Data Security in AI-Generated Code

Scenario

AI tools can sometimes generate insecure authentication logic.

Task Description

Use an AI tool to generate a **simple login system** in Python.

Analyze the generated code to check:

- Whether credentials are hardcoded
- Whether passwords are stored or compared in plain text
- Whether insecure logic is used

Then, revise the code to improve security (e.g., avoid hardcoding, use input validation).

Expected Output

- AI-generated login code
- Identification of security risks
- Revised secure version of the code
- Brief explanation of improvements



```
username = "admin"
password = "admin123"

input_username = input("Enter username: ")
input_password = input("Enter password: ")

if input_username == username and input_password == password:
    print("Login successful")
else:
    print("Login failed")
```

```
... Enter username: admin
Enter password: admin123
Login successful
```

Identified Security Risks

- Credentials are **hardcoded** in the source code.
- Passwords are stored and compared in **plain text**.
- No input validation or password protection mechanisms.
- Vulnerable to credential exposure and brute-force attacks

Revised Secure Version

```
import hashlib

users = {
    "admin": hashlib.sha256("admin123".encode()).hexdigest()
}

def login():
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()

    hashed_password = hashlib.sha256(password.encode()).hexdigest()

    if username in users and users[username] == hashed_password:
        print("Login successful")
    else:
        print("Invalid credentials")

login()

... Enter username: admin
Enter password: admin123
Login successful
```

Explanation of Improvements

- Removed hardcoded plain-text passwords.
- Used **SHA-256 hashing** to protect passwords.
- Added basic input validation.
- Improved confidentiality and reduced security risks.

Task 2: Bias Detection in AI-Generated Decision Systems

Scenario

AI systems may unintentionally introduce bias.

Task Description

Use AI prompts such as:

- *“Create a loan approval system”*
- Vary applicant names and genders in prompts

Analyze whether:

- The logic treats certain genders or names unfairly
- Approval decisions depend on irrelevant personal attributes

Suggest methods to reduce or remove bias.

AI-Generated Loan Approval Code

```
 def loan_approval(name, gender, income):  
    if gender == "male" and income > 30000:  
        return "Loan Approved"  
    elif gender == "female" and income > 50000:  
        return "Loan Approved"  
    else:  
        return "Loan Rejected"
```

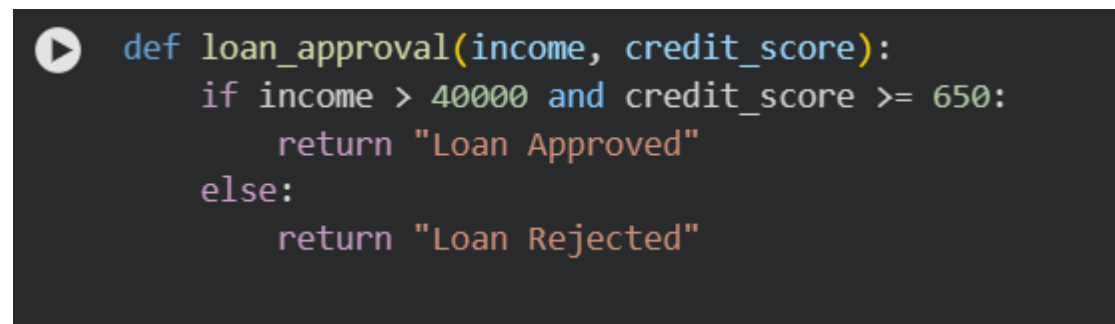
Identification of Biased Logic

- Different income thresholds based on **gender**.
- Gender is an **irrelevant attribute** for loan approval.
- Leads to unfair treatment and discrimination.

Fairness Issues

- Violates principles of equality and fairness.
- Reinforces societal bias.
- Decisions are not based solely on financial capability.

Bias Mitigation Strategy

A code block with a dark background and light-colored text. It contains a Python function named 'loan_approval' that takes 'income' and 'credit_score' as arguments. The function has an if-statement: 'if income > 40000 and credit_score >= 650:'. If true, it returns 'Loan Approved'. Otherwise, it returns 'Loan Rejected'.

```
def loan_approval(income, credit_score):  
    if income > 40000 and credit_score >= 650:  
        return "Loan Approved"  
    else:  
        return "Loan Rejected"
```

Mitigation Methods

- Remove gender and name from decision logic.
- Use objective financial indicators only.
- Regularly audit decision rules for bias.

- **Task 3: Transparency and Explainability in AI-Generated Code (Recursive Binary Search)**

- **Scenario**

- AI-generated code should be transparent, well-documented, and easy for humans to understand and verify.

- **Task Description**

- Use an AI tool to generate a Python program that:
 - Implements Binary Search using recursion
 - Searches for a given element in a sorted list
 - Includes:
 - Clear inline comments
 - A step-by-step explanation of the recursive logic

- After generating the code, analyze:
- Whether the explanation clearly describes the base case and recursive case
- Whether the comments correctly match the code logic
- Whether the code is understandable for beginner-level students

```
def binary_search(arr, target, low, high):
    """
    ... Recursive Binary Search Function
    ...
    """
    ... # Base case: element not found
    if low > high:
        return -1

    mid = (low + high) // 2

    # If target is found at mid
    if arr[mid] == target:
        return mid

    # If target is smaller, search left half
    elif arr[mid] > target:
        return binary_search(arr, target, low, mid - 1)

    # If target is larger, search right half
    else:
        return binary_search(arr, target, mid + 1, high)
```

Step-by-Step Explanation

1. **Base Case:** If $low > high$, the element does not exist.
2. **Recursive Case:**
 - Compare target with middle element.
 - Recur on left or right half accordingly.
3. Continues until the element is found or search space is empty.

Student Assessment

- Base and recursive cases are clearly defined.
- Comments correctly match code behavior.
- Logic is easy to follow for beginner-level students.
- Transparent and explainable implementation.

Task 4: Ethical Evaluation of AI-Based Scoring Systems

Scenario

AI-generated scoring systems can influence hiring decisions.

Task Description

Ask an AI tool to generate a **job applicant scoring system** based on features such as:

- Skills
- Experience
- Education

Analyze the generated code to check:

- Whether gender, name, or unrelated features influence scoring

Whether the logic is fair and objective

```
 def score_applicant(name, gender, skills, experience, education):  
    score = 0  
  
    if gender == "male":  
        score += 5  
  
    score += skills * 2  
    score += experience * 3  
  
    if education == "Masters":  
        score += 5  
  
    return score
```

Potential Bias Identified

- Gender directly affects scoring.
- Name and gender are unrelated to job performance.
- Unethical for hiring decisions.

Ethical Analysis

- Violates fairness and equal opportunity.
- Encourages discrimination.
- Lacks objective evaluation standards.

Improved Ethical Version

```
def score_applicant(skills, experience, education):  
    score = skills * 2 + experience * 3  
  
    if education == "Masters":  
        score += 5  
    elif education == "PhD":  
        score += 7  
  
    return score
```

Task 5: Inclusiveness and Ethical Variable Design

Scenario

Inclusive coding practices avoid assumptions related to gender, identity, or roles and promote fairness in software design.

Task Description

Use an AI tool to generate a Python code snippet that processes user or employee details.

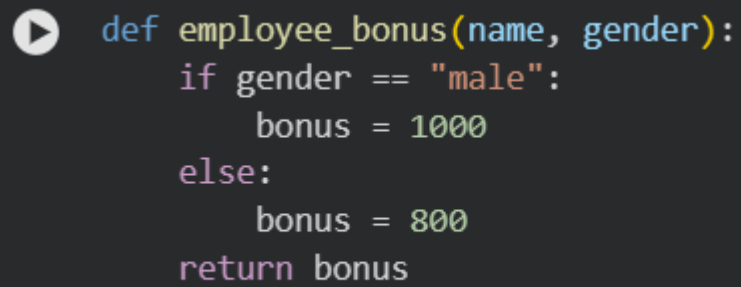
Analyze the code to identify:

- Gender-specific variables (e.g., male, female)
- Assumptions based on gender or identity
- Non-inclusive naming or logic

Modify or regenerate the code to:

- Use gender-neutral variable names
- Avoid gender-based conditions unless strictly required
- Ensure inclusive and respectful coding practices

Original AI-Generated Code

A code editor window with a dark background. On the left side, there is a circular icon with a white play button symbol. The code is written in a syntax-highlighted style: 'def' is blue, 'employee_bonus' is yellow, 'name' and 'gender' are green, 'if' is purple, 'gender == "male"' is orange, 'bonus = 1000' is green, 'else:' is purple, 'bonus = 800' is green, and 'return bonus' is purple.

```
def employee_bonus(name, gender):  
    if gender == "male":  
        bonus = 1000  
    else:  
        bonus = 800  
    return bonus
```

Issues Identified

- Gender-based assumptions.
- Non-inclusive logic.
- Reinforces stereotypes.

Explanation of Improvements

- Removed gender-specific variables.
- Used performance-based criteria.
- Promotes inclusiveness and fairness.
- Aligns with ethical software design principles.